毕业设计 · 全栈项目 · 独立交付

PetMate 智能宠物管家
把养宠的所有事，装进一个平台

一个覆盖 **用户端 / 商家端 / 平台管理后台** 三端的宠物综合服务平台。宠物主在一个系统内完成 **买用品、约服务、聊健康、晒日常、找领养**；商家完成经营与履约；平台完成治理与风控。

> 从需求拆解、原型设计、数据建模到功能实现与测试验收，独立走通全流程。本册包含 26 个高保真原型、38 张真实系统运行截图、一份可直接照读的 8 分钟演示脚本。

3 端 用户 / 商家 / 管理	**26 个** 高保真原型	**101 个** 前端页面
64 张 数据库表	**15 个** 业务模块	**11 项** 审计缺陷已修复

CONTENTS

目录

建议阅读顺序：先看 01-02 了解项目与我的角色，再看 03-04 的原型与实拍，最后用 05 的演示脚本和 09 的问答准备面试。

01 **项目定位：这个项目在解决什么问题** …… 6 个真实痛点与解法

02 **我的角色与工作方式** …… 全流程分工 + 技术边界说明

03 **高保真原型 • 26 个** …… 三端页面，23 个已落地

04 **真实系统运行截图 • 38 张** …… 三端逐页实拍

05 **现场演示脚本 • 8 分钟版** …… 逐句口播稿 + 检查清单

06 **功能全景与设计亮点** …… 15 个业务模块

07 **技术架构 • 工程规模 • 安全设计** …… 三层架构 + 实测数据 + 上线加固清单

08 **工程质量与已知限制** …… 11 项缺陷修复记录

09 **面试高频问题应答要点** …… 6 题，可直接背

本册怎么用：面试前通读 01、02、09 三节；现场演示按 05 的动线走；03、04 两节是最直观的能力证明，可以直接翻给面试官看。全部截图与原型均来自真实交付物，非示意图。

01 / POSITIONING

这个项目在解决什么问题 6 个痛点 • 6 个解法

养宠家庭的日常事务分散在电商 App、宠物医院小程序、社区群、领养帖子之间，信息割裂、履约不透明、健康记录无处沉淀。PetMate 用一套账号体系把这几件事缝合到一起。

痛点 01

买粮、约洗护、找医院要开三个 App

商品、服务、医疗分属不同平台，账号不通、优惠券不通用、订单记录各自为政。

解法 统一账号 + 统一订单中心，商品与服务共用一套购物车、优惠券与售后。

痛点 02

宠物健康记录靠纸质本子，到期没人提醒

疫苗、驱虫、体检周期容易遗忘，就医时也说不清历史病史。

解法 宠物档案 + 健康时间线，疫苗/驱虫/体检到期自动生成站内提醒。

痛点 03

宠物不舒服时，不知道该不该去医院

深夜呕吐、软便，网上搜索结果互相矛盾，去医院又怕小题大做。

解法 AI 智能问诊给出「紧急 / 建议就医 / 日常照护」分级建议，并说明判断依据。

痛点 04

寄养和上门服务，服务质量全靠运气

价格不透明、无实景、出问题没有凭证，纠纷难以追溯。

解法 服务规格透明化（体型 × 毛长分档计价）+ 评价与商家回复闭环 + 全流程留痕。

痛点 05

领养信息散落在各个微信群，真假难辨

送养信息无审核、无协议、领养后无人回访，动物福利无法保障。

解法 领养信息平台审核 + 申请与状态流转 + 领养后回访，形成可追溯的闭环。

痛点 06

门店有服务能力，但缺线上经营工具

小店依赖微信群接单，排班、库存、流水全靠手工记。

解法 商家工作台提供经营看板、订单履约、服务与评价管理，开箱即用。

02 / MY ROLE

我在这个项目里做了什么

这一节说得直白一些，因为面试里最容易被追问的，就是「你到底做了哪部分」。

角色定位：产品设计与交付执行

我承担的是**产品设计与交付执行**的角色：定义这个系统要解决什么问题、拆成哪些模块、每个模块的数据怎么流转、每个状态应该怎么跳转，然后把它们变成可运行的东西，最后验证并演示出来。

完整走通的链路是：**需求拆解 → 原型设计 → 数据建模 → 功能实现 → 测试验收 → 演示讲解**。其中编码实现环节我借助 AI 编程工具完成，我负责把关「做什么、为什么这样做、做成什么样算合格」——具体包括业务规则定义、状态机设计、边界情况枚举、验收自查，以及反复的调试与返工。

六步工作流

1
需求拆解
把「养宠一站式」拆成 15 个业务模块、3 类角色的权限边界

2
原型设计
产出 26 个高保真原型，覆盖三端全部核心流程

3
数据建模
设计 64 张表，定义实体关系与冗余聚合字段

4
功能实现
借助 AI 工具完成编码，逐模块验证

5
测试验收
源码审计 + 端到端回归，修复 11 项缺陷

6
演示讲解
整理演示路径与话术，能完整讲清产品逻辑

关于技术边界，我选择说清楚

⚠ 这部分建议照着说

Spring Boot、Vue3、JWT 这些框架的底层原理**不是我的强项**，我不会把它们写成简历上的「精通」。

我能做到的是：讲清每个业务模块的设计取舍、每个状态机的流转规则、每个边界情况为什么这样处理，并且**现场演示出来**。如果你需要的是能独立设计业务系统、把需求落成可用产品、并能对客户讲清楚产品价值的人，这个项目可以证明我具备这个能力。

03 / PROTOTYPE

高保真原型 • 26 个 26 个

所有页面在设计阶段先出高保真原型，确认信息层级与操作路径后，再进入开发。其中 23 个原型已落地为真实可运行页面，3 个后台设计稿作为后续迭代储备。



首页 已落地

原型目录 UI/stitch/petmate_1

落地 → /



商品分类 已落地

原型目录 UI/stitch/_19

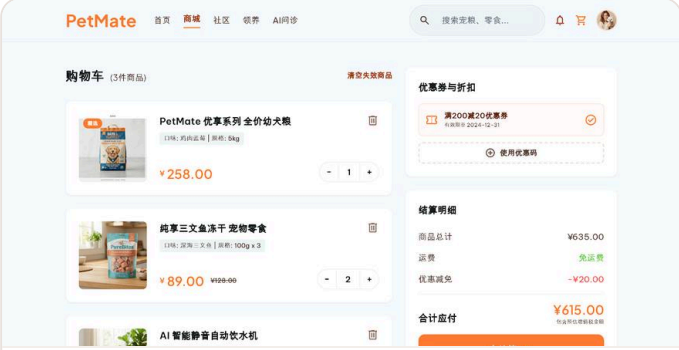
落地 → /category



本地服务 已落地

原型目录 UI/stitch/_15

落地 → /service、/service/:id



购物车 已落地

原型目录 UI/stitch/_14

落地 → /cart、/checkout



我的订单 已落地

原型目录 UI/stitch/_7

落地 → /order/list



宠物档案 已落地

原型目录 UI/stitch/_6

落地 → /pet/health/:id

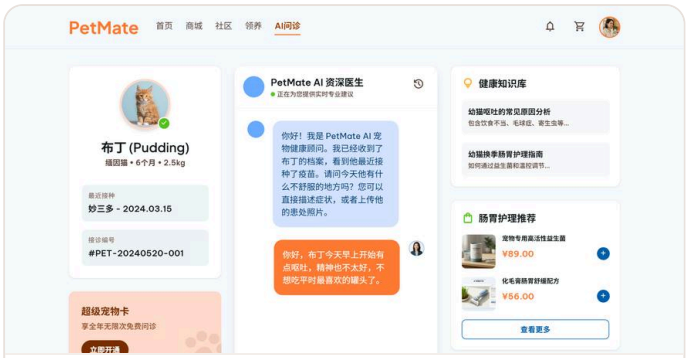
高保真原型



宠物档案 · 空状态 已落地

原型目录 UI/stitch/_9

落地 → `/pet/edit/:id`



AI 智能问诊 已落地

原型目录 UI/stitch/ai

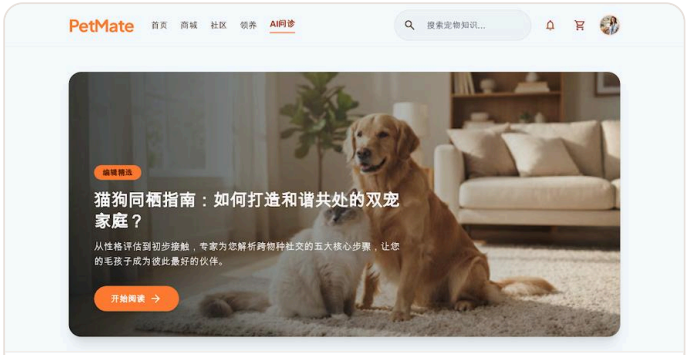
落地 → `/consult`



社区动态 已落地

原型目录 UI/stitch/_22

落地 → `/community`



宠友百科 已落地

原型目录 UI/stitch/_1

落地 → `/knowledge`



个人中心 已落地

原型目录 UI/stitch/_5

落地 → `/user`



登录 / 注册 已落地

原型目录 UI/stitch/petmate_2

落地 → `/login`、`/register`

高保真原型



商家中心登录 已落地

原型目录 UI/stitch/_17

落地 → `/merchant/landing`



商家工作台 已落地

原型目录 UI/stitch/_12

落地 → `/merchant/dashboard`



经营总览 Dashboard 已落地

原型目录 UI/stitch/_13

落地 → `/merchant/data/sales`



数据看板 v3.13.6 已落地

原型目录 UI/stitch/_3

落地 → `/admin/dashboard`（跨端复用）



订单处理中心 已落地

原型目录 UI/stitch/_16

落地 → `/merchant/order`



订单与商品管理 已落地

原型目录 UI/stitch/_10

落地 → `/merchant/product/list`

高保真原型



商品与服务管理 已落地

原型目录 UI/stitch/_21

落地 → `/merchant/service/list`



评价管理 已落地

原型目录 UI/stitch/_2

落地 → `/merchant/review`



系统登录 已落地

原型目录 UI/stitch/_11

落地 → `/admin/login`



用户管理 已落地

原型目录 UI/stitch/_20

落地 → `/admin/user/list`



内容审核中心 已落地

原型目录 UI/stitch/_4

落地 → `/admin/community/post`



系统配置 设计探索稿

原型目录 UI/stitch/_8

待后续迭代

高保真原型



订单与商品管理 设计探索稿

原型目录 UI/stitch/_18

待后续迭代



风险控制与安全中心 设计探索稿

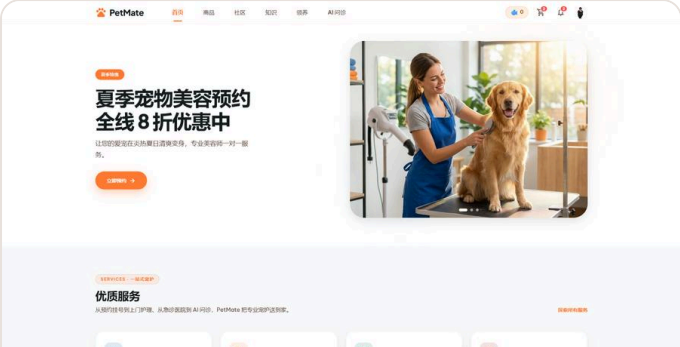
原型目录 UI/stitch/_23

待后续迭代

04 / LIVE DEMO

真实系统运行截图 · 38 张 38 个

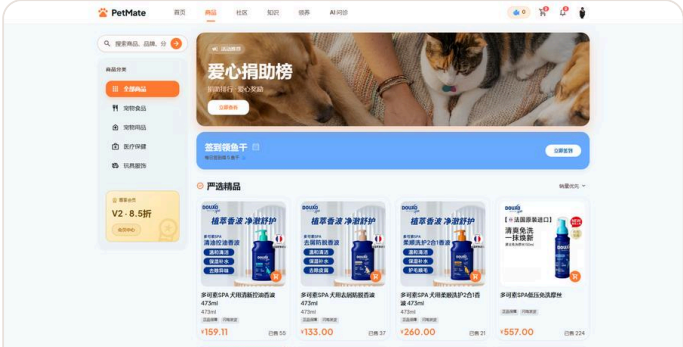
以下截图全部来自真实运行的系统：MySQL 建库、Spring Boot 后端、Vue3 前端三层同时拉起，用真实演示账号登录后逐页截取——不是原型图，也不是静态 Mock。



首页

品牌入口 + 本地服务四入口 + 为你精选推荐流 + 签到领鱼干

/user 端



商品分类

三级分类树 + 限时秒杀主视觉 + 严选精品卡片墙

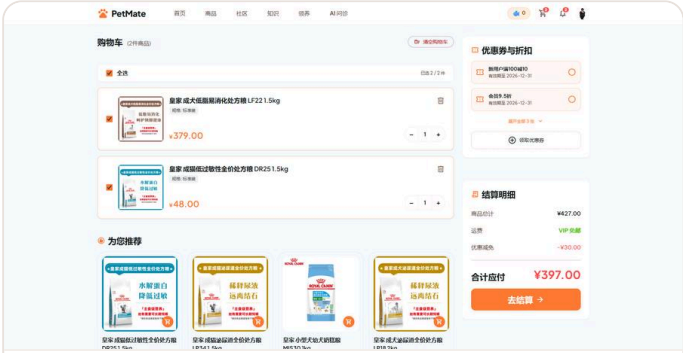
/user 端



商品详情

多规格 SKU 选择、富文本详情、评价与商家回复、看了又看

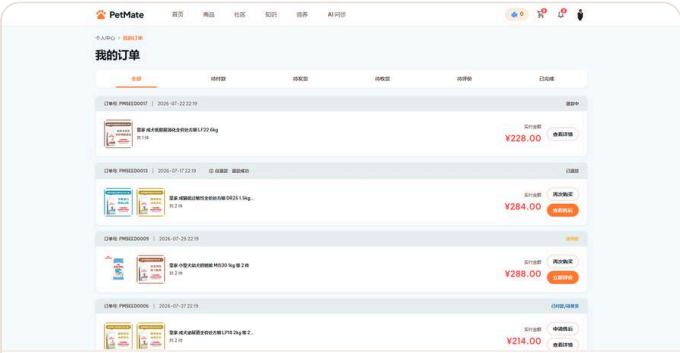
/user 端



购物车

勾选结算、数量增减、优惠券与折扣、结算明细

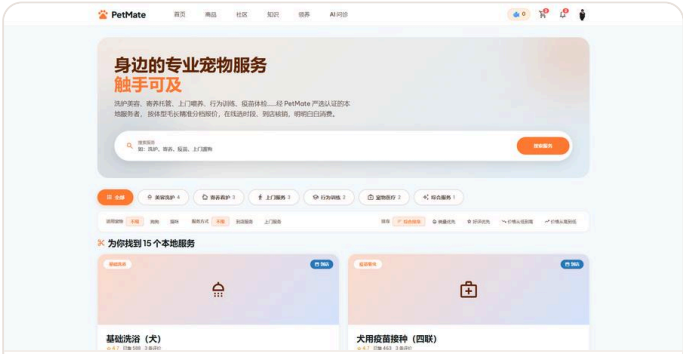
/user 端



我的订单

五态分栏：全部 / 待付款 / 待发货 / 待收货 / 待评价

/user 端

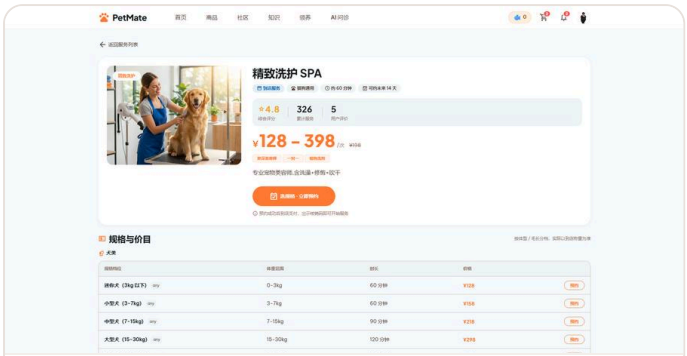


本地服务

服务列表 + 筛选排序 + 到店 · 上门两种履约方式

/user 端

真实系统运行截图



服务详情

规格分档（体型 × 毛长）、加购项、容量选择、预约下单

`/user 端`



健康档案

宠物健康时间线：体重 / 疫苗 / 驱虫 / 体检 / 病史

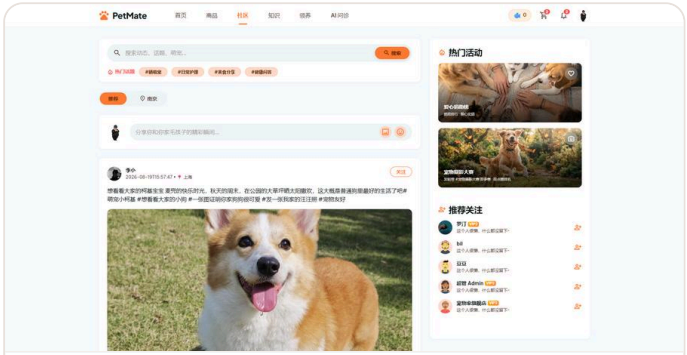
`/user 端`



AI 智能问诊

接智谱 GLM-4-Flash 真实大模型，SSE 流式输出，多轮对话

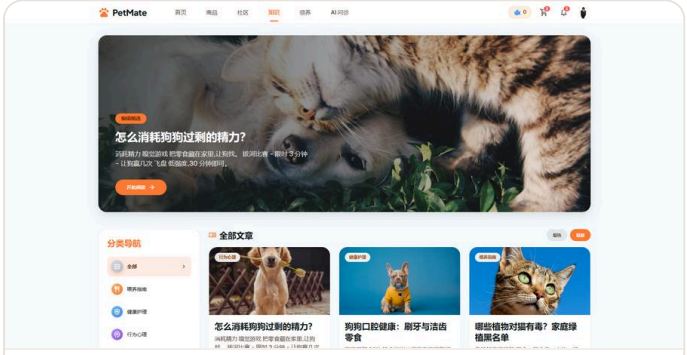
`/user 端`



社区动态

图文 / 短视频发帖、话题标签、点赞评论关注

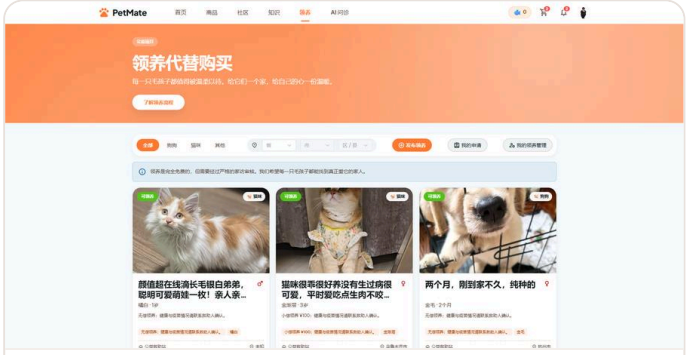
`/user 端`



宠友百科

知识库分类导航、最热 / 最新排序、文章分页

`/user 端`

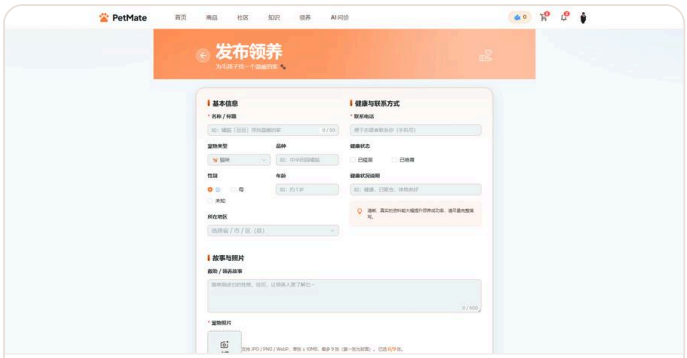


领养救助

领养信息列表、品种与地区筛选、详情查看

`/user 端`

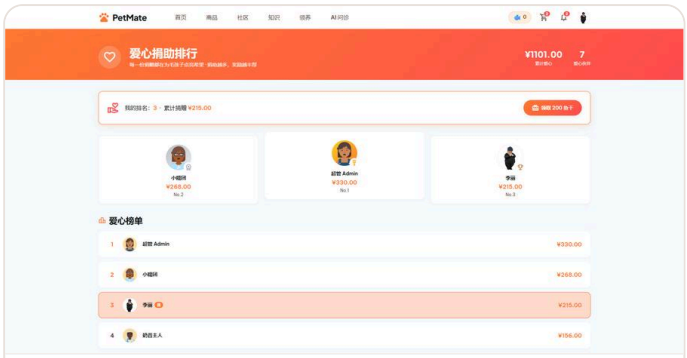
真实系统运行截图



发布领养

逐张上传模型：即时预览 / 独立进度 / 失败重试（9 张 · 10MB）

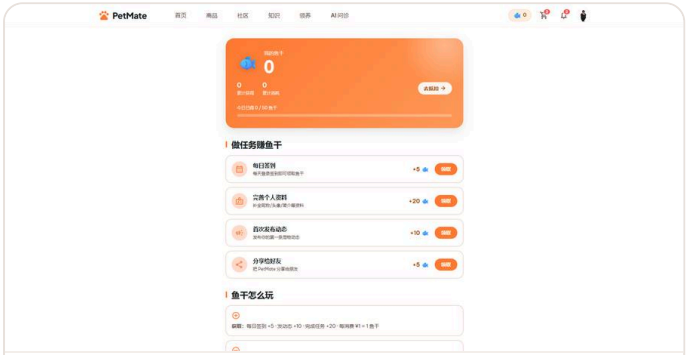
`/user 端`



爱心捐助排行

公益捐助榜单 + 鱼干认领，平台公益属性

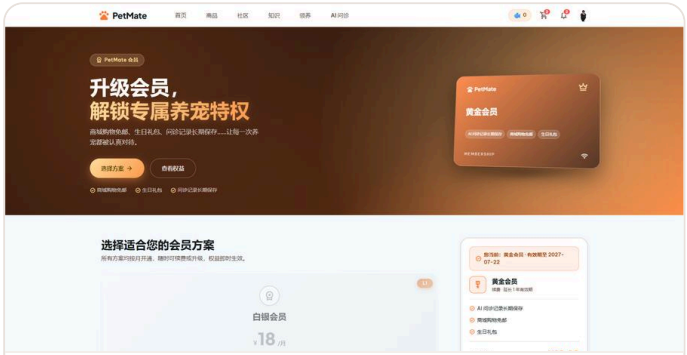
`/user 端`



任务中心

鱼干积分任务体系（每日上限 50，10 鱼干 = ¥1，抵扣封顶 10%）

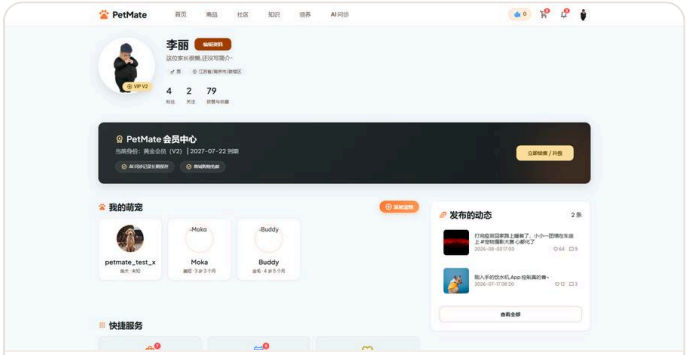
`/user 端`



开通会员

VIP 套餐选择 + 支付结果轮询 + 权益说明

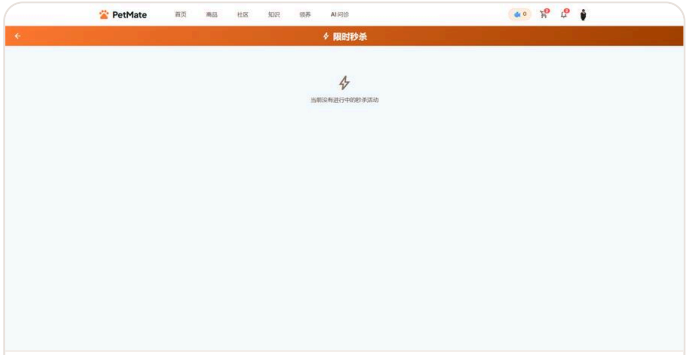
`/user 端`



个人中心

用户资料、会员中心、我的萌宠、快捷服务入口

`/user 端`

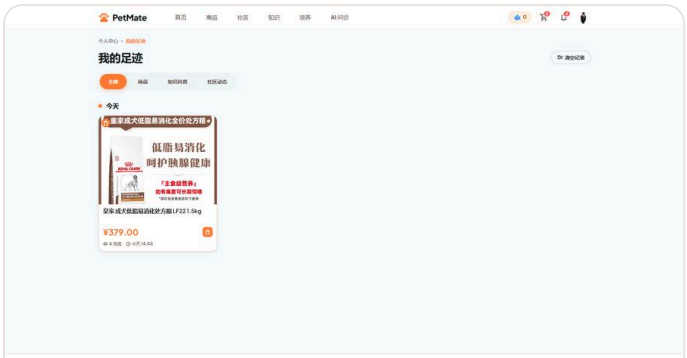


限时秒杀

秒杀场次 + CAS 原子扣库存 + 限购 1 件 + 失败回补

`/user 端`

真实系统运行截图



商品足迹

全链路浏览足迹，行为数据回流推荐

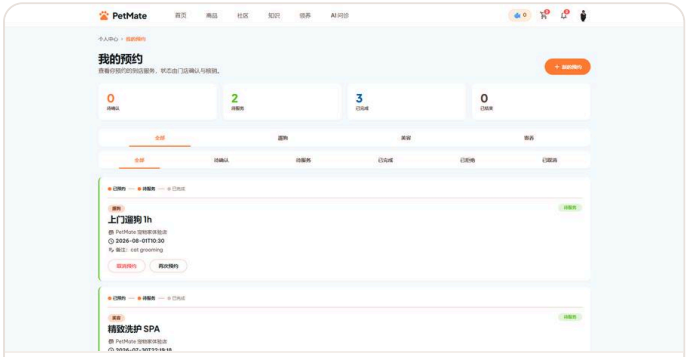
`/user 端`



我的售后

售后单列表与状态流转（11 态状态机）

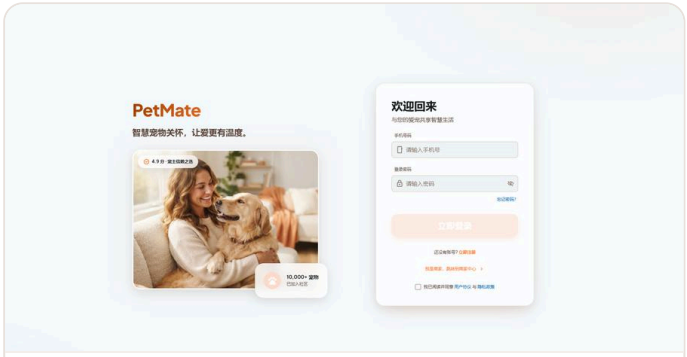
`/user 端`



我的预约

服务预约记录与核销状态

`/user 端`



登录页

手机号 + 密码登录，JWT 令牌签发

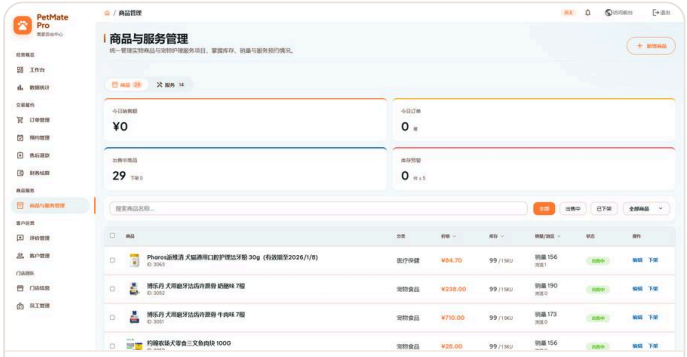
`/user 端`



商家工作台

今日销售额 / 订单量 / 待办事项，经营一屏掌握

`/merchant 端`

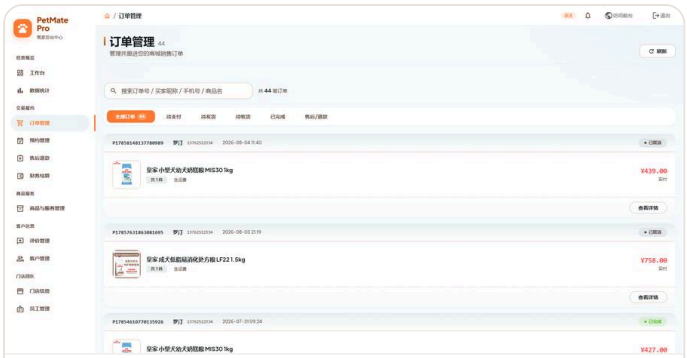


商品管理

商品列表、上下架、价格库存维护、删除保护（子角色校验）

`/merchant 端`

真实系统运行截图



订单管理

订单履约：发货、核销、异常订单标记

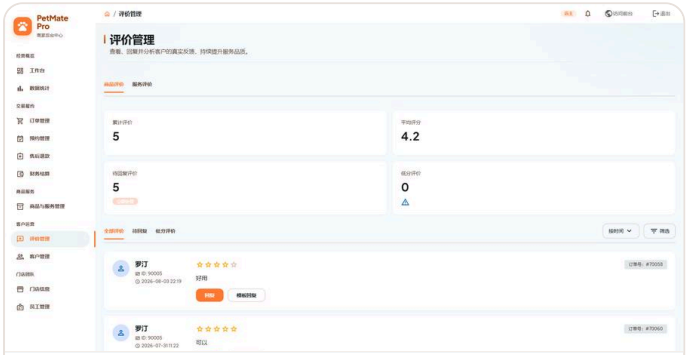
`/merchant 端`



服务管理

服务上下架、规格与价格、容量配置

`/merchant 端`



评价管理

待回复 / 低分 / 申诉筛选 + 商家公开回复闭环

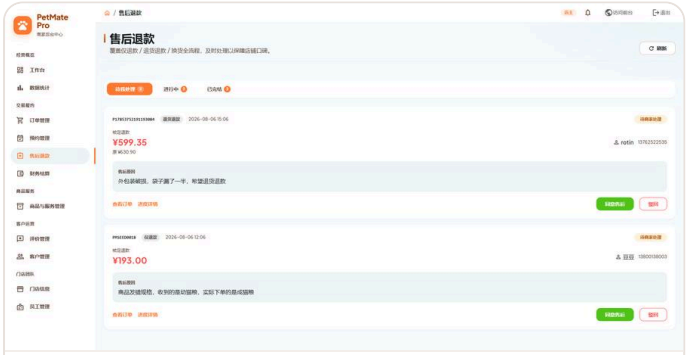
`/merchant 端`



销售看板

销售趋势、客流来源、转化漏斗、热销排行

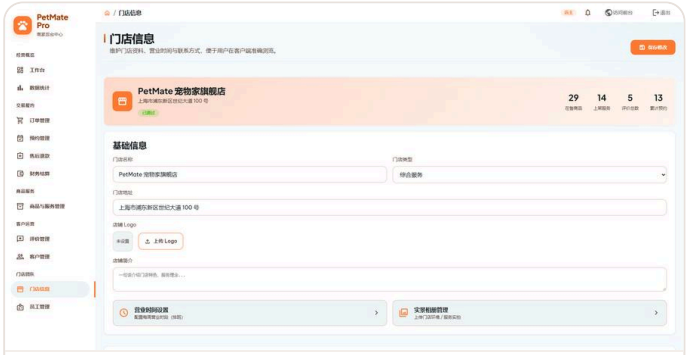
`/merchant 端`



售后退款

退款申请处理与凭证上传

`/merchant 端`

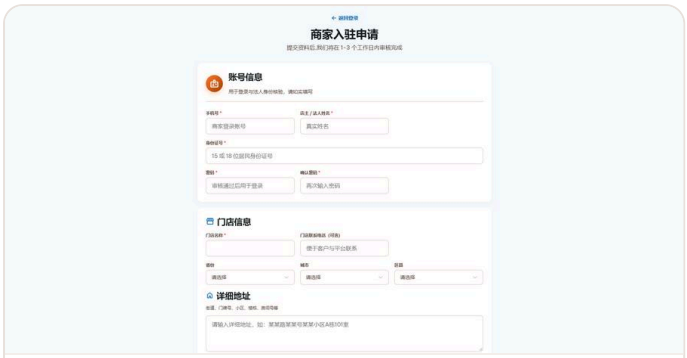


门店信息

店铺资料维护、实景相册（部分更新白名单）

`/merchant 端`

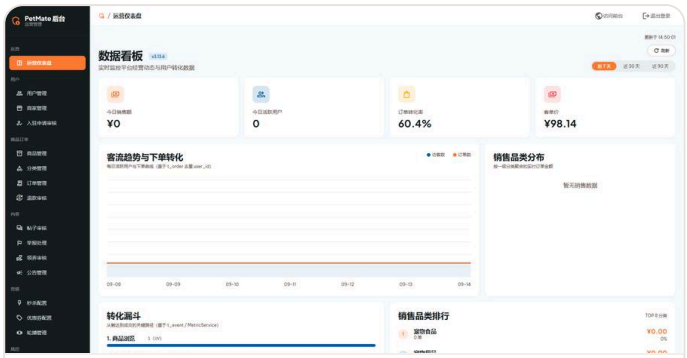
真实系统运行截图



申请入驻

商家入驻申请与审核状态查询

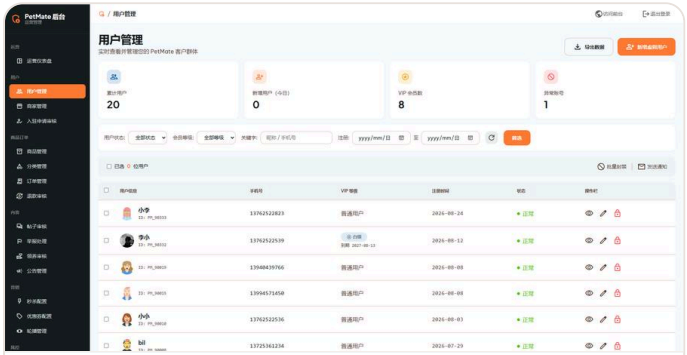
`/merchant 端`



运营仪表盘

平台核心指标可视化：用户数、订单数、GMV、投诉率

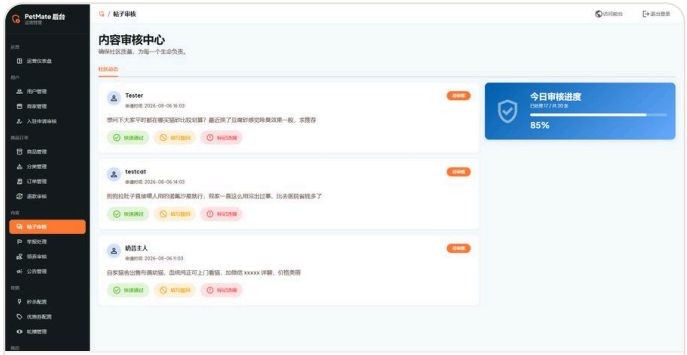
`/admin 端`



用户管理

用户列表与筛选、详情、封禁 / 解封

`/admin 端`



内容审核

社区动态 / 知识科普 / 领养救助三队列审核

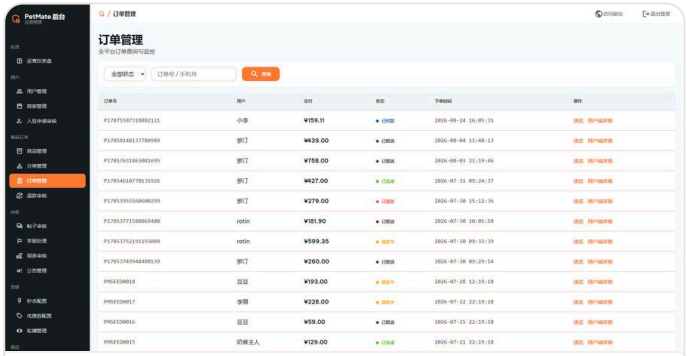
`/admin 端`



风控中心

举报处理中心 + 关键词过滤 + 处置留痕

`/admin 端`

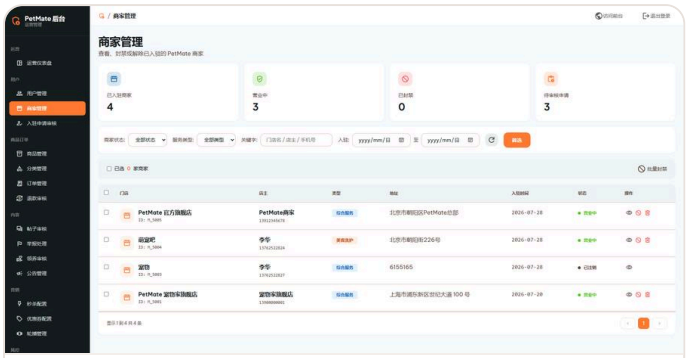


订单管理

全平台订单监控与人工介入

`/admin 端`

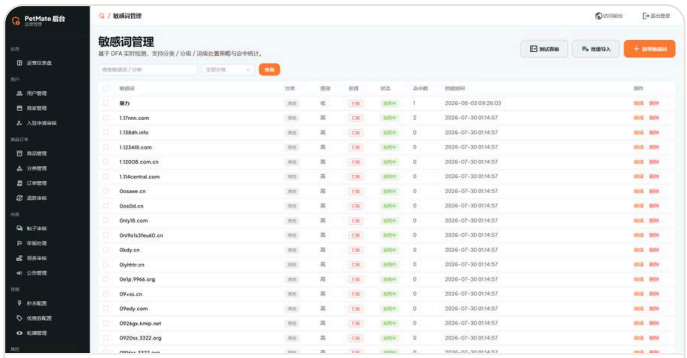
真实系统运行截图



商家管理

商家列表 + 入驻申请审核

`/admin 端`



敏感词管理

敏感词库维护，社区内容发布前置拦截

`/admin 端`

05 / DEMO SCRIPT

现场演示脚本 · 8 分钟版

如果面试时让你「演示一下」，照着这个顺序走，全程约 8 分钟，主线清晰、不卡壳。

演示账号 密码统一 123456

角色	手机号	可做的事	入口
普通用户	`13800138000`	购物下单、服务预约、AI 问诊、宠物档案、社区、领养	`/login`
商家	`13900000001`	经营看板、商品/服务管理、订单履约、评价回复、售后处理	`/merchant/dashboard`
平台管理员	`13700000000`	用户治理、内容审核、风控中心、订单监控、敏感词管理	`/admin/dashboard`

演示动线（1 / 2）

00:00
30 秒

开场：一句话讲清这是什么

"PetMate 是一个宠物综合服务平台，养宠的人在这里能买用品、约洗护寄养、记健康档案、用 AI 问诊，还能发社区和做领养。它有三个端——用户端、商家端和平台后台。"

00:30
2 分钟

① 购物主干链路（用户端）

"从首页开始——这里是服务入口和推荐流。进商品分类，选一个商品看详情：注意这里有规格选择，不同规格价格和库存是分开的。加购 → 打开 /cart 购物车 → 勾选商品 → 展示优惠券与结算明细 → 提交订单 → 跳到 /order/list 看订单状态。订单有完整状态机：待付款、待发货、待收货、待评价，超时未支付会由定时任务自动取消并回补库存。"

02:30
1 分钟

② 服务预约（区别于普通电商的地方）

"服务不是标品，所以我把规格拆成了体型 × 毛长两档，再加可选的加购项和上门费。最终金额是「规格单价 × 天数 + 加购项 + 上门费」，寄养按天算。"

03:30
1 分钟

③ AI 智能问诊

"这块接的是真实大模型（智谱 GLM-4-Flash），不是写死的问答。它会给出分级建议——紧急、建议就医、日常照护，并解释为什么这么判断。"

演示动线（2 / 2）

04:30 1 分钟	**④ 宠物档案与健康提醒** "每只宠物一个档案，健康记录按时间线排：体重、疫苗、驱虫、体检、病史。系统会在疫苗到期前自动发站内提醒。"
05:30 1.5 分钟	**⑤ 商家端：经营与履约** "切到商家账号。工作台是今天卖了多少、有多少单要处理。订单管理里能发货核销；评价管理能看到待回复和低分评价，商家可以直接公开回复。"
07:00 1 分钟	**⑥ 管理端：治理与风控** "后台是平台视角。内容审核有三个队列——社区动态、知识科普、领养救助。风控中心处理举报，还有敏感词库做发布前置拦截。"
08:00	**收尾：把话题引回「我能做什么」** "整个系统是三层架构，前端 Vue3、后端 Spring Boot、数据库 MySQL，我自己主导的产品设计和业务建模，编码借助 AI 工具完成。如果您想看某个模块更细的设计，我可以直接展开讲。"

本地启动命令

```
# 1. 初始化数据库（MySQL 8.0，root / 123456）
cd petmate-backend/sql && ./reset_and_seed.sh

# 2. 启动后端（JDK 21）
cd petmate-backend && ./mvnw spring-boot:run  # 监听 8080

# 3. 启动前端
cd frontend/web && npm install --legacy-peer-deps && npm run dev  # 5173
```

05 / DEMO SCRIPT

演示前检查清单

现场演示翻车基本都栽在这几件事上。建议演示前一天按这份清单过一遍。

① 服务是否都在跑

MySQL（3306）→ 后端（8080）→ 前端（5173）。后端和前端要分别开两个终端，别关掉。

② 用哪个账号演示

提前三个浏览器窗口分别登录用户、商家、管理员，切换时不输密码。

③ 数据是否可见

购物车里要有商品、订单列表要有记录、健康档案要有数据，别演示空列表。

④ 网络断了怎么办

AI 问诊需要联网。如果现场网络不稳，改用作品集里的截图讲，不要现场等。

⑤ 兜底材料

这份作品集和原型演示站都是单文件 HTML，U 盘里放一份，断网也能翻。

⑥ 别硬撑技术细节

被问到不熟的实现细节，直接说「这块是 AI 辅助实现的，我讲一下它的业务规则」，比编更安全。

06 / HIGHLIGHTS

功能全景与设计亮点

系统按 15 个业务模块组织，覆盖三类角色的完整日常操作。下面是几个在业务规则上做过认真推敲的地方。

AI 智能问诊

接入智谱 GLM-4-Flash 真实大模型，SSE 流式输出。输出分三级：紧急 / 建议就医 / 日常照护，并解释判断依据。无法确诊时明确提示就医，不给误导性结论。带每日配额控制。

真实 LLM · 流式输出

售后 11 态状态机

售后不是简单的「申请—同意」，而是 11 个状态、三条角色分支（用户 / 商家 / 平台仲裁）的完整状态机，每次流转都写入日志表，可追溯每一步由谁在什么时候操作。

状态机 · 全链路留痕

限时秒杀

库存扣减用 CAS 原子更新（带库存充足条件），配合每人限购 1 件。下单失败或订单超时未支付时，回补秒杀名额、SKU 库存、优惠券与积分，不会超卖或库存泄漏。

CAS 原子扣减 · 失败回补

鱼干积分体系

签到、评价、发帖等任务产出鱼干，每日上限 50。10 鱼干抵 1 元，单笔订单抵扣封顶 10%。积分流水可追溯，任务防重复。

任务去重 · 抵扣封顶

领养救助闭环

送养发布（逐张图片上传、即时预览、失败可单张重试）→ 平台审核 → 用户申请 → 状态流转 → 领养后回访。送养人只能管理自己发布的信息，越权被服务端拦截。

归属校验 · 上传容错

宠物关怀提醒

定时任务每分钟扫描提醒表，疫苗 / 驱虫 / 体检到期前按提前天数生成站内消息。支持一次性与周期两种模式，完成或删除后不再重复推送。

定时调度 · 站内信

15 个业务模块 15 个模块

按业务域组织，覆盖用户端、商家端、平台管理后台三类角色的全部日常操作。

01
用户中心
注册登录、个人资料、收货地址、会员等级

02
宠物档案
基础信息、健康记录、成长日记、智能提醒

03
商品商城
三级分类、商品详情、搜索、评价、收藏足迹

04
购物车与订单
购物车、下单四步、订单状态机、发票

05
支付与钱包
多种支付方式、优惠券、余额、退款

06
AI 智能问诊
多轮对话、分级建议、关联商品推荐

07
美容洗护与训练
门店、技师、套餐、时段选择

08
寄养托管
家庭 / 门店寄养、寄养期间跟踪、接回流程

09
社区动态
图文短视频发布、互动、推荐、内容审核

10
领养救助
送养信息、领养申请、状态流转、回访

11
知识科普
分类内容、收藏点赞、行为回流推荐

12
营销活动
限时秒杀、优惠券、积分、节日活动

13
商家后台
工作台、商品、服务、订单、评价、数据看板

14
平台管理后台
仪表盘、用户、审核、风控、权限、配置

15
推荐系统
行为埋点、画像、多路召回、首页 feed

07 / ARCHITECTURE

技术架构

前后端分离的三层架构，三个前端应用共享同一套后端 API 与数据库，按角色下发不同的界面与权限。

客户端层

用户端 Web
响应式 · 桌面 / 平板 / 手机

商家工作台 PetMate Pro
PC 端

平台管理后台
PC 端 · 运营 / 客服 / 财务 / 超管

▼ /api/v1/* 统一响应 {code, msg, data, traceId} ▼

应用层 • Spring Boot 3.3 (JDK 21)

鉴权与安全
JWT 过滤器 · 令牌上下文 · BCrypt · 幂等键防重放

业务模块 ×24
商品 / 订单 / 支付 / 售后 / 秒杀 / 积分 / 宠物 / 社区 / 问诊 / 领养 / 预约 / 服务 / 商家 / 管理

通用能力
统一响应与错误码 · 链路追踪 · 敏感词过滤 · 定时任务

▼ JDBC ▼

数据层 • MySQL 8.0 (utf8mb4) • 64 张表

用户与权限 • 宠物与健康
t_user · t_pet · t_pet_health · t_pet_reminder

商品与交易
t_product · t_product_sku · t_cart · t_order · t_order_item · t_refund_log

社区与内容 • 营销 • 商家
t_post · t_article · t_seckill · t_coupon · t_store · t_service_item

07 / ARCHITECTURE

技术选型与工程规模

以下为实测统计值，非估算。项目文档早期记录的「27 张表、60+ 页面」为设计阶段数字，实际交付规模以此页为准。

技术选型

层	选型	说明
前端	Vue 3.5 + TypeScript + Vite 5 + Element Plus + Pinia + Vue Router + ECharts	1920 设计稿，postcss 转 vw 做视口适配
后端	Spring Boot 3.3 + JDK 21 + Spring JDBC + jjwt + BCrypt	24 个业务模块、33 个 Controller
数据库	MySQL 8.0（utf8mb4）	64 张表，覆盖全部业务域
AI 能力	智谱 GLM-4-Flash	AI 问诊，SSE 流式返回
接口规范	RESTful，统一响应 `{code, msg, data, traceId}`	错误码分段，便于前端统一处理

工程规模 实测数据

101 个 前端 .vue 页面文件	**102 条** 前端路由（含三端角色守卫）	**33 个** 后端 Controller	**64 张** 数据库表
340 演示用户数据	**74 / 54** 商品 / 订单数据	**48 / 51** 社区动态 / 领养信息	**26 → 23** 原型 → 已落地页面

接口与数据规范

统一响应格式

`{ code, msg, data, traceId }`

code=0 为成功，其余按业务域分段：10001 参数错误、20001 未登录、30001 库存不足、40001 订单不存在、60001 敏感词命中、70001 AI 繁忙、90001 无权限。

鉴权与角色

除登录注册外均需 `Authorization: Bearer <token>`；JWT 无状态令牌，密码 BCrypt 加密存储；商家子账号通过请求头区分门店范围，实现数据隔离。

07 / ARCHITECTURE

安全设计与上线加固

安全设计覆盖认证、授权、防滥用、内容安全、数据隔离与边界校验六个方面；右侧是生产环境上线前必须补齐的项。

安全设计

认证

JWT 令牌 + BCrypt 密码存储，登录态 7 天自动续期。

授权

基于角色的访问控制，接口级校验；商家只能访问自己门店数据。

防滥用

关键写操作采用幂等键防重放；秒杀限购 1 件。

内容安全

敏感词库前置拦截 + 社区内容审核队列 + 用户一键举报。

数据隔离

普通用户只能查看自己的订单、宠物与售后记录。

边界校验

归属校验（assertOwner）贯穿领养、地址、评价等写操作。

上线前加固清单 生产环境必做

密钥与账号

JWT secret 换为 ≥32 字节强随机串；MySQL 改用专用账号并限制权限。

去掉内存态依赖

引入 Redis 替代内存版幂等锁与限流，支持多实例部署。

接真实能力

接入真实支付通道（含回调验签）与生产级 LLM 配额管理。

传输与跨域

启用 HTTPS，配置 CORS 白名单，关闭调试端点。

可观测性

接入日志聚合（ELK / Loki）与监控告警（Prometheus + Grafana）。

部署编排

Docker Compose 编排 MySQL / Redis / Backend / Frontend / Nginx。

08 / QUALITY

工程质量：11 项审计缺陷修复记录

项目做过一轮风险导向的源码审计，逐条读代码验证边界情况，发现的问题全部修复并回归。这部分想证明的是：我有检查自己交付物的习惯。

后端 • 7 项

后端 **下单扣库存非原子** — 高并发下存在超卖风险，改为带库存条件的 CAS 原子更新。

后端 **商品详情缺少上架状态过滤** — 已下架商品仍可通过直链访问，补齐状态过滤与响应字段。

后端 **下单后购物车整单清空** — 未下单的商品也被删除，改为只清实际成交项。

后端 **商家删除商品缺少子角色校验** — 补充权限校验，防止越权删除。

后端 **评价回复链路未闭环** — 商家回复未在用户侧回显，补齐回复字段与提交事务。

后端 **商品举报入口未接入** — 举报类型白名单补充商品类型。

后端 **创建订单与提交评价缺少事务** — 补充事务注解，避免部分写入。

前端 • 4 项

前端 **购物车无勾选 UI 与结算拦截** — 补齐勾选交互，未勾选商品时阻止结算。

前端 **商品列表使用了假评价数** — 改为读取真实评价数。

前端 **规格选择未做字典化** — 改为规格字典映射，避免串档。

前端 **无 SKU 商品价格显示 NaN** — 补充兜底取值。

测试资产

测试计划 5 份
总览与测试策略、后端测试计划、前端测试计划、端到端与集成测试计划、变更记录。

端到端验证脚本
基于浏览器自动化编写多场景脚本（下单、评价、领养、分享等），可重复执行并留档截图。

测试报告与覆盖率
后端单元测试覆盖率报告、前端与端到端测试结果归档。

08 / QUALITY

已知限制

主动说明限制比被追问出来更可信。以下几项是明确知道、但基于项目定位（功能完整可演示的产品原型）暂未处理的部分。

支付为演示态

支付通道走沙箱与模拟余额，不发生真实资金流。生产落地需要接入真实支付通道并处理回调验签。

积分每日上限并发非原子

每日上限校验在并发场景下理论上可被绕过，属已知的 P2 级待优化项，生产环境需要加锁或改为原子累加。

权限体系为演示态

RBAC 的角色与权限数据来自配置，接口级权限校验以演示为目的，生产环境建议引入成熟鉴权框架。

未做压测与高可用

没有做过压力测试、缓存层与集群部署，当前定位是功能完整、可运行、可演示的完整产品原型。

如果重做，我会优先改这四件事

测试前置

现在是功能做完才补测试，应该先把关键路径的验收用例写出来，边做边验。

权限模型重做

真实项目应一开始就引入成熟鉴权框架，后期改造成本很高。

支付与状态机解耦

支付回调、订单状态、售后状态现在耦合较紧，应抽成独立的状态流转服务。

加一层缓存

商品详情和分类树读多写少，应加缓存降低数据库压力。

面试高频问题应答要点（上）

这几题被问到的概率最高。建议先自己念一遍，念不顺的地方就是你还没想清楚的地方。

Q 这个项目是你自己做的吗？你具体做了哪部分？

这个项目从产品定义到上线级交付是我独立完成的。需求拆解、模块划分、数据建模、业务规则与状态机设计、原型设计、验收自查、演示讲解由我主导；编码实现环节借助 AI 编程工具完成，我负责定义需求、判断产出是否符合预期、发现问题并推动修复。

然后立刻给出证据：我能具体讲清楚每个模块为什么这么设计。比如售后为什么是 11 个状态而不是简单的「申请—同意」，因为涉及用户、商家、平台三方，平台仲裁必须是一个独立环节，否则商家拒绝后用户没有救济路径。

Q 你用了哪些技术？熟悉到什么程度？

不要吹。技术栈是 Vue3 + TypeScript + Element Plus 做前端，Spring Boot 3 + JDK 21 做后端，MySQL 8 存数据，AI 问诊接的是智谱 GLM-4-Flash。

但要**把话锋转到你的强项**：框架底层实现不是我的强项，我也不会把它写成精通。我的优势在于知道一个业务系统应该长什么样——数据怎么建模、状态怎么流转、边界情况在哪、用户在什么场景下会卡住。这个项目里 64 张表和 15 个业务模块的设计是我能完整讲清楚的部分。

Q 开发过程中遇到的最难的问题是什么？

挑一个**能讲出技术含量又不心虚**的：**秒杀和下单的库存扣减。**

最初的做法是「查出库存 → 判断够不够 → 更新库存」三步走，这个在并发下会超卖，因为判断和更新之间有间隙。后来改成一条 SQL 里带条件更新（库存大于购买数量才更新，用受影响行数判断是否成功），失败直接返回库存不足。同时处理了失败回补：下单失败或订单超时未支付时，要把秒杀名额、SKU 库存、用掉的优惠券、扣掉的积分都还回去，否则库存会「泄漏」。

这个例子好在：它证明你理解「判断和操作之间的间隙」这个并发本质，而不只是背了个方案。

面试高频问题应答要点（下）

Q 如果让你重做，你会改进什么？

这题考的是反思能力，别说「没什么要改的」。

- **测试前置**：现在是功能做完才补测试，应该先把关键路径的验收用例写出来，边做边验。
- **权限模型重做**：现在 RBAC 是演示态，真实项目应该一开始就引入成熟的鉴权框架。
- **支付与状态机解耦**：应该抽成独立的状态流转服务。
- **加一层缓存**：商品详情和分类树读多写少，应该加缓存降低数据库压力。

Q 你觉得自己更适合什么岗位？

这个项目完整走了一遍「发现需求 → 设计方案 → 落地交付 → 验证效果」的循环，我觉得自己更擅长也更喜欢靠近业务和用户的那一端：理解真实场景里的痛点、把需求翻译成可执行的方案、把产品讲清楚并演示出来。

所以售前、技术支持、产品这类需要**既懂业务逻辑又能对外讲清楚**的岗位会更适合我。这份作品集就是我的证据——它不只是代码，是一套能讲、能演示、能回答追问的完整方案。

Q 这个系统能跑起来吗？现在能演示吗？

能，而且准备了两种演示方式：

方式一 · 真实系统：MySQL + 后端 + 前端三层拉起来，用三个演示账号分别登录用户端、商家端和管理端，走完整流程。
方式二 · 离线兜底：作品集和原型演示站都是单文件 HTML，不依赖网络，断网也能翻着讲。

另外准备了 26 个高保真原型，每个都标注了它对应到哪个真实页面，可以对照着看设计意图和最终实现的差别。

一句话总结你的卖点

交付过 2 个完整产品 · 懂业务逻辑与数据建模 · 能用 AI 工具高效产出 · 能讲清并现场演示产品。

材料清单

这套材料按使用场景分成三份，本册是其中最适合随身携带的一份。

PetMate-作品集.pdf（本册）
面试通读、打印、发送给面试官；含全部原型与实拍截图

PetMate-原型演示站.html
现场演示原型走查；26 个原型支持「设计稿 / 可交互原型」双模式切换

PetMate-作品集.html
在线浏览版；与原型演示站互相跳转，适合发链接